HOW-TO

CSS print page numbers with counter(page), footers and fixed sheets

Updated 22 Dec 2026

KEY TAKEAWAYS

- CSS print page numbers go in an `@page` margin box: `content: counter(page)`, and `counter(pages)` for the total.
- MDN's table lists margin boxes in Chrome 131, Edge 131 and Opera 116, and not in Firefox or Safari.
- `counter(page)` outside a margin box printed 0 in our Chrome 157 test. The spec defines page counters for use in page-margin boxes.
- Playwright and Puppeteer fill `pageNumber` and `totalPages` elements in a `footerTemplate`, whatever the page's CSS says.
- On fixed sheets, one element per page, an ordinary CSS counter on the sheet numbers the pages without margin boxes.

CONTENTS

1 CSS print page numbers in a margin box 2
2 Page numbers in Chrome’s print dialog 3
3 Why counter(page) is not working 3
4 Page x of y, other starting numbers, no number on page one 4
5 HTML to PDF page numbers from footerTemplate 5
6 Numbers you place yourself 5
7 Page numbers on papyrus sheets 6
8 What to remember 7

For CSS print page numbers, put `counter(page)` in the `content` of an `@page` margin box, and `counter(pages)` where you want the total: `@bottom-center { content: "Page " counter(page) " of " counter(pages); }`. MDN's compatibility table lists margin boxes in Chrome 131, Edge 131 and Opera 116, and not in Firefox or Safari. Anywhere else, take the number from the PDF tool's footer, or place the numbers yourself.

Below: the margin-box version, why `counter(page)` fails, the footer templates in Playwright and Puppeteer, and numbering fixed sheets. Where pages break is in css page break print.

Our tests ran on 11 November 2026 in the installed Google Chrome 157, printed through Playwright with `channel: "chrome"`. Playwright's default headless shell is a different build, and its docs say to expect different behavior in some cases. We didn't test it or other browsers.

1. CSS print page numbers in a margin box

The CSS Paged Media draft creates a counter named `page` for you, so a page number needs no setup. It goes up by 1 on every page, and the draft says it's a real counter that `counter()` can read. A second counter, `pages`, always holds the total number of pages, and its value can't be changed. You show them in a margin box:

```
@page {
  size: A4;
  margin: 20mm 15mm;
  @bottom-center { content: "Page " counter(page) " of " counter(pages); }
}
```

On a page of 120 short paragraphs, this printed five A4 pages, numbered "Page 1 of 5" to "Page 5 of 5". Chrome's blog post of 30 October 2024 uses the `page` counter the same way and describes `pages` as the total.

The margin is part of the recipe. Chrome's blog says the height of the top and bottom boxes comes from the margin you set in `@page`, so a bottom box needs a bottom margin. With `margin: 0`, our number didn't print at all.

Left and right pages

Books put the number in the outside corner. Chrome’s blog does that with page selectors: `@page :right { @bottom-right { content: counter(page); } }` for right-hand pages and `@page :left { @bottom-left { content: counter(page); } }` for left-hand ones. In our test, odd pages got the number on the right and even pages on the left.

Browser support

MDN marks the margin boxes as experimental. We read its compatibility table in a browser on 11 November 2026 and checked it against MDN’s browser-compat-data package, version 8.1.8:

Browser	Margin boxes, so `counter(page)` there
Chrome, Chrome Android	131
Edge	131
Opera	116
Firefox	No
Safari, Safari on iOS	No

Firefox and Safari print nothing from a margin box. A script that prints with Chromium falls in the first rows, whatever browser your reader uses.

2. Page numbers in Chrome’s print dialog

A CSS print page number in Chrome can come from two places at once. Chrome’s blog says the browser adds its own header and footer when there is room, even if you added yours, and that the print dialog can turn them off. If you see two numbers, one is the browser’s.

3. Why counter(page) is not working

When “counter page” CSS isn’t working, the cause is usually one of these.

1. **It isn't in a margin box.** The draft says page counters are defined in an `@page` rule and used as content in page-margin boxes. We put `counter(page)` in a `::after` on the body, and in a fixed footer element. Chrome 157 printed `0` each time.
2. **The browser has no margin boxes.** Per MDN, Firefox and Safari print nothing from them.
3. **The margin is 0.** The box has no room, as above.
4. **The counter's name is wrong.** The predefined counter is `page`. MDN's `@page` page has a syntax example with `counter(pageNumber)`; in our test that name printed "Page 0" on every page. Chrome's blog and the draft both use `page`.
5. **There is no `content`.** The draft generates a margin box only if its `content` isn't `none`.

4. Page x of y, other starting numbers, no number on page one

For "page x of y", use both counters, as in the first example.

To start at another number, use `counter-increment`. The draft says the `page` counter goes up by 1 on every page unless `counter-increment` in the page context gives a different increment, and its own example, `@page { counter-increment: page 2; }`, makes every page even-numbered. So `@page :first { counter-increment: page 5; }` should make the first page 5. In Chrome 157 it did: the pages printed 5 to 9. The total from `counter(pages)` stayed 5, so "Page 9 of 5" is what you get if you combine them. `counter-reset: page 4` on `@page :first`, the obvious-looking alternative, printed 4, then 1, 2, 3, 4.

To hide the number on page one, give that page's box no content:

```
@page :first { @bottom-center { content: none; } }
```

The draft's `:first` matches the first printed page. Its own examples put margin boxes inside `@page :first`, one of them an empty footer on the first page. Chrome 157 left the first page blank and numbered pages 2 to 5. MDN's page on `:first` has a narrower note: it says you can change only the margins, orphans, widows and page breaks with it. That note doesn't mention margin boxes, so check the result in the browsers you print from.

5. HTML to PDF page numbers from footerTemplate

If a script makes the PDF, the tool can number the pages whatever the page's CSS says. For HTML to PDF page numbers, Playwright's `page.pdf()` has `displayHeaderFooter`, off by default, and a `footerTemplate`. Elements with the classes `pageNumber` and `totalPages` get the values, and so do `date`, `title` and `url`:

```
await page.pdf({
  path: "report.pdf",
  format: "A4",
  margin: { top: "15mm", bottom: "20mm" },
  displayHeaderFooter: true,
  headerTemplate: "<span></span>",
  footerTemplate: '<div style="font-size:9px;width:100%;text-align:center">Page
<span class="pageNumber"></span> of <span class="totalPages"></span></div>',
});
```

On a page with no `@page` rules, that gave "Page 1 of 4" to "Page 4 of 4". The template is a small document of its own: Playwright's docs say scripts in it don't run and the page's styles don't reach it, so style it inline. The full script, and the other options, are in Playwright: generate PDF from HTML.

Puppeteer's `PDFOptions` use the same option names, and its `footerTemplate` has, in its docs' words, "the same constraints and support for special classes" as the header. Gotenberg's Chromium routes take a `footer.html` file in which elements with the classes `pageNumber` and `totalPages` get the values, printed in the bottom margin.

6. Numbers you place yourself

On fixed sheets, where each page is one element, you don't need the browser to know the page number: you do. An ordinary CSS counter on the sheet element works without margin boxes, because it counts elements, not pages. MDN's guide to counters describes `counter-reset` and `counter-increment` for numbering elements like this, and MDN's data lists `counter()` in Chrome, Edge, Firefox and Safari.

```
<style>
  @page { size: A4; margin: 0; }
  body { margin: 0; counter-reset: sheet; }
  .sheet { width: 210mm; height: 297mm; position: relative; overflow: hidden;
           counter-increment: sheet; break-after: page; }
  .sheet::after { content: "Page " counter(sheet) " of " var(--total);
                  position: absolute; bottom: 10mm; right: 15mm; font: 9pt sans-
serif; }
</style>
<section class="sheet"><h1>One</h1></section>
<section class="sheet"><h1>Two</h1></section>
<section class="sheet"><h1>Three</h1></section>
<script>
  const sheets = document.querySelectorAll(".sheet");
  document.body.style.setProperty("--total", `"${sheets.length}"`);
</script>
```

In Chrome 157 this printed three pages, “Page 1 of 3” to “Page 3 of 3”. The total comes from a line of script, because a counter only knows the sheets up to the current one. It holds only while one sheet really is one page, and that’s the part to check. Sheet sizes are in css page size a4.

A table of contents with page numbers is harder. In CSS it takes `target-counter()`, which reads the page number of a link’s target. Chrome’s blog lists cross-references like that under “future possibilities”, and Chrome 157 printed nothing for it in our test. In Chromium, those page numbers have to come from you.

7. Page numbers on papyrus sheets

papyrus, a toolkit by n-uf for documents rendered as HTML and exported as PDF, is built on fixed sheets. Each element with the class `pdf-page` is one sheet on screen and one page in the PDF, and the exporter won’t write a PDF whose page count disagrees with the page elements or the `pageCount` you expect. That check is what makes self-placed numbers safe: a sheet that spilled onto an extra page would shift every number after it, and the export stops instead.

For a table of contents, `numberToc()` numbers the entries. It returns a map from entry id to `"1."`, `"2."`, `"A."` for an appendix, or `null` for an entry it skips. Those are entry numbers, not page numbers. In `@n-uf/papyrus-react`, `TOCItem` takes `number`, `title` and `page` props, so the page a heading sits on is a value you pass, and you can check it because you know which sheet it's on. The html to pdf a4 page has more.

papyrus is young. Its versions are dates (`YY.M.R`), not semantic versions, so any release may break things. Pin an exact version and read the CHANGELOG before you upgrade. papyrus and n-uf are not affiliated with Microsoft's Playwright project, the Puppeteer or Gotenberg projects, or the browser makers named here.

8. What to remember

- Put `counter(page)` and `counter(pages)` inside an `@page` margin box, and give the page a margin to hold it.
- MDN lists margin boxes in Chrome 131, Edge 131 and Opera 116, not in Firefox or Safari.
- Outside a margin box, `counter(page)` printed 0 in Chrome 157. The counter is called `page`, not `pageNumber`.
- A `footerTemplate` with `pageNumber` and `totalPages` doesn't depend on the page's CSS.
- On fixed sheets, a counter on the sheet numbers the pages as long as one sheet is one page.

Questions people ask

How do I show "page x of y" on a printed page?

Put `counter(page)` and `counter(pages)` in the `content` of a margin box: `@bottom-center { content: "Page " counter(page) " of " counter(pages); }`. The CSS Paged Media draft says `pages` always holds the total number of pages. It works where margin boxes do, which per MDN is Chrome 131, Edge 131 and Opera 116 and later.

Can page numbers start at a number other than 1?

In Chrome 157 they could, with `counter-increment` on the first page. The draft says an explicit `counter-increment` for `page` in the page context replaces the default increment of 1, and `@page :first { counter-increment: page 5; }` printed 5 on the first page and 6 on the next in our test. The total from `counter(pages)` stayed the real page count. `counter-reset: page 4` on `@page :first` printed 4, then 1, 2, 3, 4, so don't use that.

How do I hide the page number on the first page?

Give the first page's margin box no content: `@page :first { @bottom-center { content: none; } }`. The draft generates a margin box only if its `content` isn't `none`, and its own examples put margin boxes inside `@page :first`. In Chrome 157 the first page had no number and the others did. MDN's `:first` page has a narrower note, so check the browsers you print from.

Why is counter(page) showing 0 or nothing?

Usually because it isn't in a margin box, the name is wrong, or the margin is 0. In our Chrome 157 test, `counter(page)` in a `::after` on the body printed 0, `counter(pageNumber)` in a margin box printed 0, and a margin box on a page with `margin: 0` printed nothing. The draft uses page counters in page-margin boxes, and the predefined counter is called `page`.

How do I add page numbers when I make a PDF with a script?

Use the PDF tool's footer. In Playwright, set `displayHeaderFooter: true` and a `footerTemplate` with elements of the classes `pageNumber` and `totalPages`. Puppeteer's `footerTemplate` has the same special classes, and Gotenberg's `footer.html` uses them too.

Sources

1. MDN: @page at-rule (page source and compatibility table read in Chrome) ↗ checked 11 Nov 2026
2. MDN browser-compat-data 8.1.8 (npm, published 2026-11-02, the latest on 2026-11-11) ↗ checked 11 Nov 2026
3. W3C: CSS Paged Media Module Level 3 (Working Draft, 14 September 2023) ↗ checked 11 Nov 2026
4. Chrome for Developers: Add content to the margins of web pages when printed using CSS (30 October 2024) ↗ checked 11 Nov 2026
5. MDN: :first ↗ checked 11 Nov 2026
6. MDN: Using CSS counters ↗ checked 11 Nov 2026
7. Playwright: Page, page.pdf() ↗ checked 11 Nov 2026
8. Playwright: Browsers (Chromium headless shell and new headless mode) ↗ checked 11 Nov 2026
9. Puppeteer: PDFOptions ↗ checked 11 Nov 2026
10. Gotenberg: Convert HTML to PDF (9.x) ↗ checked 11 Nov 2026
11. papyrus README (@n-uf/papyrus v26.11.1) ↗ checked 11 Nov 2026
12. papyrus-react README (@n-uf/papyrus-react v26.11.1) ↗ checked 11 Nov 2026